CONDICIONAIS

As condicionais permitem que você, o programador, permita que seu programa tome decisões: como se seu programa pudesse escolher entre pegar o caminho da esquerda ou o caminho da direita com base em certas condições.

Construído no Python está um conjunto de “operadores” que são usados para fazer perguntas matemáticas.

Os símbolos > e < provavelmente são bastante familiares para você.

>= denota “maior ou igual a”.

<= denota “menor ou igual a”.

== denota “igual, mas observe o duplo sinal de igual! Um único sinal de igual atribuiria um valor. Sinais duplos de igual são usados para comparar variáveis.

!= denota “diferente de.

As declarações condicionais comparam um termo à esquerda com um termo à direita.

DECLARAÇÕES

Na janela do terminal, digite o código compare.py. Isso criará um novo arquivo chamado “comparar”.
Na janela do editor de texto, comece com o seguinte:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x < y:
```

```
    print("x is less than y")
```

Observe como seu programa recebe a entrada do usuário para xey, convertendo-os como números inteiros e salvando-os em suas respectivas variáveis xey. Então, a instrução if compara x e y. Se a condição x < y for atendida, a instrução print será executada.

As instruções if usam valores booleanos ou booleanos (verdadeiro ou falso) para decidir se devem ou não ser executadas. Se a afirmação de x > y for verdadeira, o compilador irá registrá-la como verdadeira e executar o código.

FLUXO DE CONTROLE, ELIF E MAIS

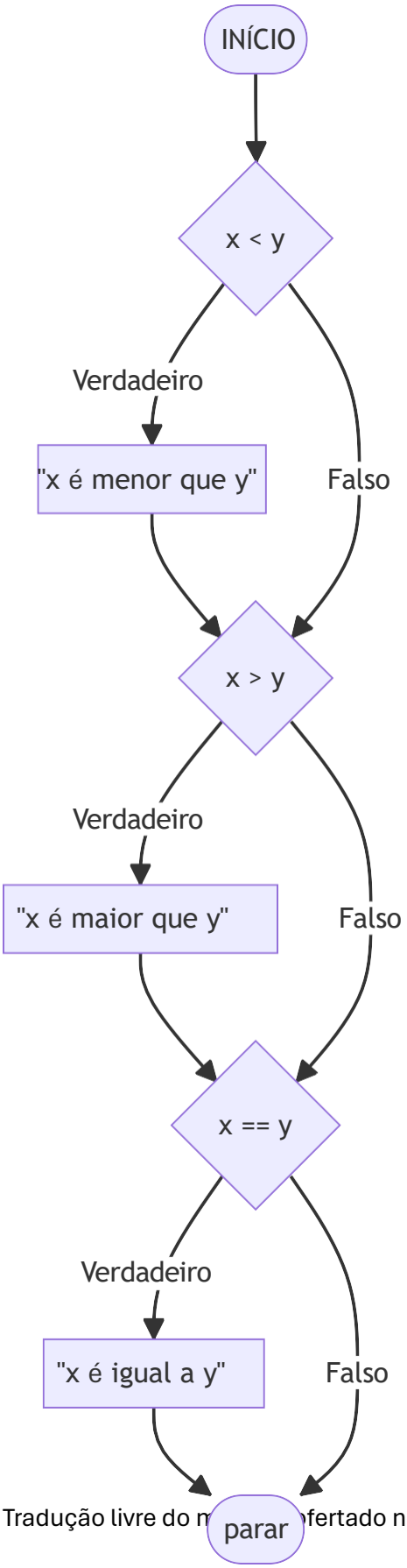
Revise ainda mais seu código da seguinte maneira:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x < y:
    print("x is less than y")
if x > y:
    print("x is greater than y")
if x == y:
    print("x is equal to y")
```

Observe como você está fornecendo uma série de instruções if. Primeiro, a primeira instrução if é avaliada. Então, a segunda instrução if executa sua avaliação. Finalmente, a última instrução if executa sua avaliação. Esse fluxo de decisões é chamado de “fluxo de controle”.

Nosso código pode ser representado da seguinte forma:


INÍCIO
x < y
Verdadeiro
"x é menor que y"
Falso
x > y
Verdadeiro
"x é maior que y"
Falso
x == y
Verdadeiro
"x é igual a y"
Falso
parar

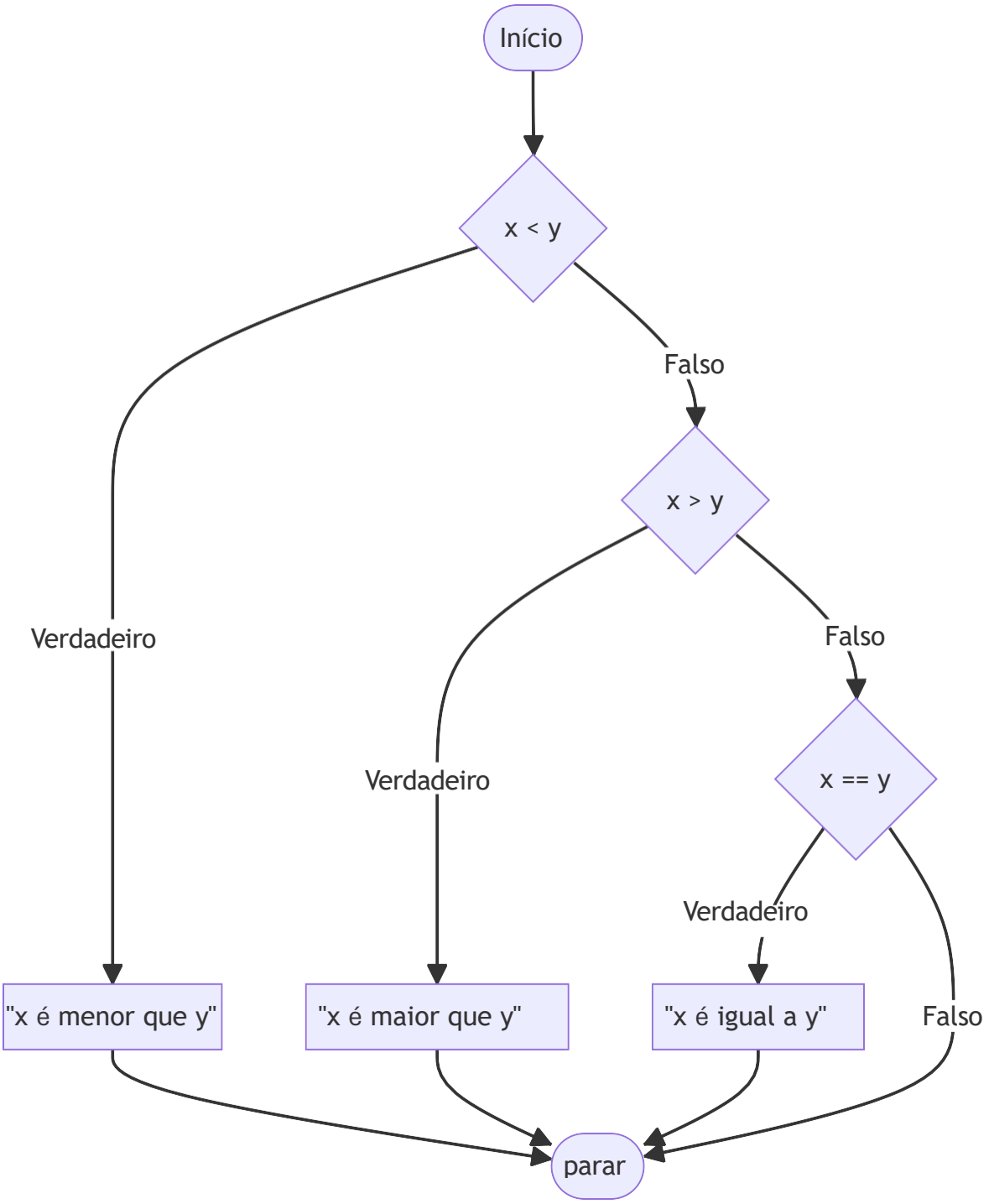
Este programa pode ser melhorado não fazendo três perguntas consecutivas. Afinal, nem todas as três questões podem ter um resultado verdadeiro! Revise seu programa da seguinte forma:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x < y:
  print("x is less than y")
elif x > y:
  print("x is greater than y")
elif x == y:
  print("x is equal to y")
```

Observe como o uso de elif permite que o programa tome menos decisões. Primeiro, a instrução if é avaliada. Se esta afirmação for verdadeira, todas as instruções elif não serão executadas. No entanto, se a instrução if for avaliada e considerada falsa, o primeiro elif será avaliado. Se isso for verdade, não será executada a avaliação final.

Nosso código pode ser representado da seguinte forma:



Embora seu computador possa não notar uma diferença em termos de velocidade entre nosso primeiro programa e este programa revisado, considere como um servidor online executando bilhões ou trilhões

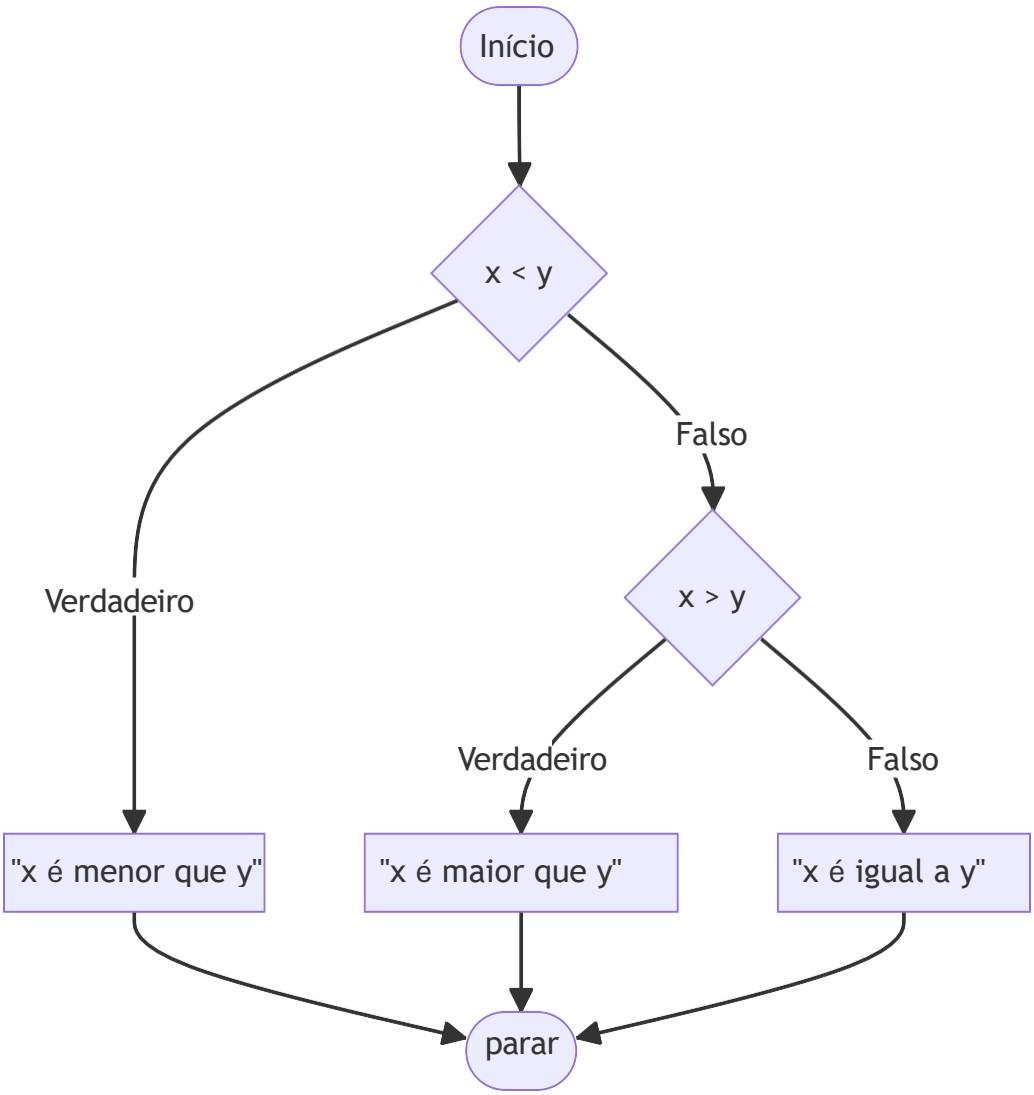
desses tipos de cálculos todos os dias poderia definitivamente ser impactado por uma decisão de codificação tão pequena.
Há uma melhoria final que podemos fazer em nosso programa. Observe como logicamente elif x == y não é uma avaliação necessária para ser executada. Afinal, se logicamente x não é menor que y E x não é maior que y, x DEVE ser igual a y. Portanto, não precisamos executar elif x == y. Podemos criar um resultado padrão “pega-tudo” usando uma instrução else. Podemos revisar da seguinte forma:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x < y:
  print("x is less than y")
elif x > y:
  print("x is greater than y")
else:
  print("x is equal to y")
```

Observe como a complexidade relativa deste programa diminuiu durante a nossa revisão.

Nosso código pode ser representado da seguinte forma:



OR

O Or permite que seu programa decida entre uma ou mais alternativas. Por exemplo, poderíamos editar ainda mais nosso programa da seguinte maneira:

```
x = int(input("What's x? "))

y = int(input("What's y? "))


if x < y or x > y:
```

```
    print("x is not equal to y")
else:
    print("x is equal to y")
```

Observe que o resultado do nosso programa é o mesmo, mas a complexidade diminui. A eficiência do nosso código é aumentada.

Neste ponto, nosso código é muito bom. No entanto, o design poderia ser melhorado ainda mais? Poderíamos editar ainda mais nosso código da seguinte maneira:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x != y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```

Observe como removemos ou inteiramente e simplesmente perguntamos: “X não é igual a y?” Fazemos uma e apenas uma pergunta. Muito eficiente!

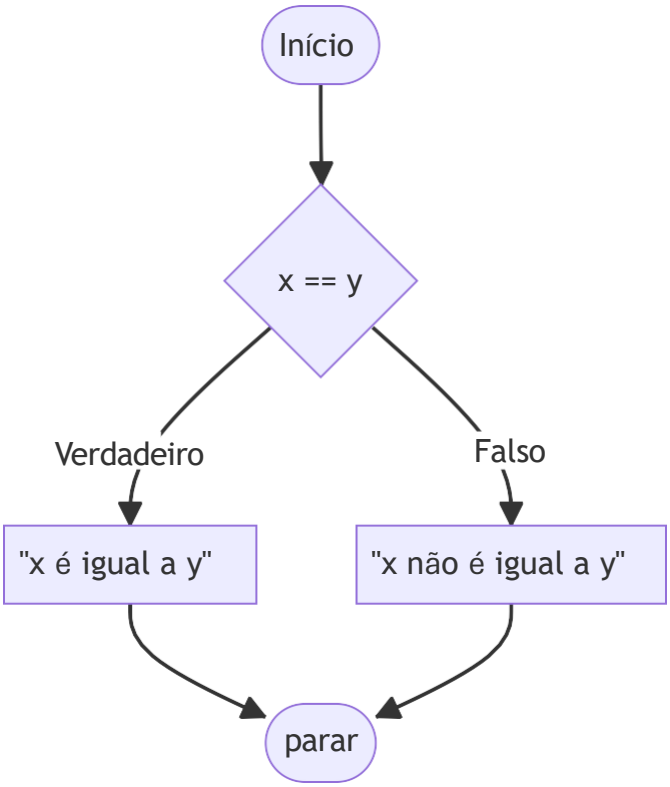
Para fins de ilustração, também poderíamos alterar nosso código da seguinte forma:

```
x = int(input("What's x? "))
y = int(input("What's y? "))

if x == y:
    print("x is equal to y")
else:
    print("x is not equal to y")
```

Observe que o operador == avalia se o que está à esquerda e à direita são iguais entre si. O uso de sinais duplos de igual é muito importante. Se você usar apenas um sinal de igual, provavelmente será gerado um erro pelo compilador.

Nosso código pode ser ilustrado da seguinte forma:



AND

Semelhante ao or, o and pode ser usado em declarações condicionais.

Execute na janela do terminal o código grade.py. Inicie seu novo programa da seguinte maneira:

```
score = int(input("Score: "))

if score >= 90 and score <= 100:
    print("Grade: A")
elif score >=80 and score < 90:
```

```
    print("Grade: B")
elif score >=70 and score < 80:
    print("Grade: C")
elif score >=60 and score < 70:
    print("Grade: D")
else:
    print("Grade: F")
```

Observe como o Python permite encadear os operadores e as condições de uma forma bastante incomum em outras linguagens de programação.

Ainda assim, podemos melhorar ainda mais nosso programa:

```
score = int(input("Score: "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

MODULO

Em matemática, paridade refere-se ao fato de um número ser par ou ímpar.

O operador módulo% na programação permite ver se dois números se dividem uniformemente ou se dividem e têm resto.

Por exemplo, 4% 2 resultaria em zero, porque divide uniformemente. No entanto, 3 % 2 não se divide uniformemente e resultaria num número diferente de zero!

Na janela do terminal, crie um novo programa digitando o código parity.py. Na janela do editor de texto, digite seu código da seguinte forma:

```
x = int(input("What's x? "))

if x % 2 == 0:
  print("Even")
else:
  print("Odd")
```

Observe como nossos usuários podem digitar qualquer número 1 ou maior para ver se é par ou ímpar.

CRIANDO NOSSA PRÓPRIA FUNÇÃO DE PARIDADE

Conforme discutido na Aula 0, você achará útil criar sua própria função!

Podemos criar nossa própria função para verificar se um número é par ou ímpar. Ajuste seu código da seguinte maneira:

```
def main():
  x = int(input("What's x? "))
  if is_even(x):
    print("Even")
  else:
```

```
    print("Odd")


def is_even(n):
  if n % 2 == 0:
    return True
  else:
    return False


main()
```

Observe que nossa instrução if is_even(x) funciona mesmo que não haja nenhum operador nela. Isso ocorre porque nossa função retorna um bool (ou booleano), verdadeiro ou falso, de volta à função principal. A instrução if simplesmente avalia se is_even of x é verdadeiro ou falso.

PYTHONIC

No mundo da programação, existem tipos de programação que são chamados de “Pythonic” por natureza. Ou seja, existem formas de programar que às vezes só são vistas na programação Python. Considere a seguinte revisão do nosso programa:

```
def main():
  x = int(input("What's x? "))
  if is_even(x):
    print("Even")
  else:
    print("Odd")
```

```
def is_even(n):
    return True if n % 2 == 0 else False


main()
```

Observe que esta declaração de retorno em nosso código é quase como uma frase em inglês. Esta é uma forma única de codificação vista apenas em Python.

Podemos revisar ainda mais nosso código e torná-lo cada vez mais legível:

```
def main():
    x = int(input("What's x? "))
    if is_even(x):
        print("Even")
    else:
        print("Odd")


def is_even(n):
    return n % 2 == 0


main()
```

Observe que o programa avaliará o que está acontecendo em n % 2 == 0 como verdadeiro ou falso e simplesmente retornará isso para a função principal.

MATCH

Semelhante às instruções if, elif e else, as instruções match podem ser usadas para executar condicionalmente código que corresponda a determinados valores.

Considere o seguinte programa:

```
name = input("What's your name? ")

 if name == "Harry":
    print("Gryffindor")
 elif name == "Hermione":
    print("Gryffindor")
 elif name == "Ron":
    print("Gryffindor")
 elif name == "Draco":
    print("Slytherin")
 else:
    print("Who?")
```

Observe que as três primeiras declarações condicionais imprimem a mesma resposta.

Podemos melhorar um pouco este código com o uso da palavra-chave ou:

```
name = input("What's your name? ")

 if name == "Harry" or name == "Hermione" or name == "Ron":
    print("Gryffindor")
 elif name == "Draco":
    print("Slytherin")
 else:
    print("Who?")
```

Observe que o número de instruções elif diminuiu, melhorando a legibilidade do nosso código.

Alternativamente, podemos usar declarações de correspondência para mapear nomes para casas. Considere o seguinte código:

```
name = input("What's your name? ")

match name:
    case "Harry":
        print("Gryffindor")
    case "Hermione":
        print("Gryffindor")
    case "Ron":
        print("Gryffindor")
    case "Draco":
        print("Slytherin")
    case _:
        print("Who?")
```

Observe o uso do símbolo _ no último caso. Isso corresponderá a qualquer entrada, resultando em um comportamento semelhante ao de uma instrução else.

Uma instrução match compara o valor após a palavra-chave match com cada um dos valores após as palavras-chave case. Caso uma correspondência seja encontrada, a respectiva seção de código recuada é executada e o programa interrompe a correspondência.

Podemos melhorar o código:

```
name = input("What's your name? ")

match name:
    case "Harry" | "Hermione" | "Ron":
        print("Gryffindor")
    case "Draco":
        print("Slytherin")
```

```
    case _:
        print("Who?")
```

Observe o uso da barra vertical única |. Muito parecido com a palavra-chave or, isso nos permite verificar vários valores na mesma instrução case.

RESUMINDO

Agora você tem o conhecimento em Python de usar instruções condicionais para fazer perguntas e fazer com que seu programa aja de acordo. Nesta aula discutimos…

- Condicionais;
- Declarações;
- Fluxo de controle, elif, and else;
- or;
- and;
- Modulo;
- Criando nossa propria função de paridade;
- Pythonic;
- match.